

The screenshot shows the GeeksforGeeks website interface for the 'Triplet Sum in Array' problem. The left sidebar indicates the problem is solved successfully with 1111/1111 test cases passed, 4/4 points scored, and a time taken of 1.97. The main area displays the Python code for the solution, which uses a two-pointer approach after sorting the array.

```
1 class Solution:
2     def hasTripletSum(self, arr, target):
3         arr.sort()
4         n = len(arr)
5
6         for i in range(n - 2):
7             lo, hi = i + 1, n - 1
8             while lo < hi:
9                 s = arr[i] + arr[lo] + arr[hi]
10                if s == target: return True
11                elif s < target: lo += 1
12                else: hi -= 1
13
14        return False
```

Logic Used: Sort the array, then fix one element at a time using a loop. For each fixed element $arr[i]$, use two pointers (lo and hi) starting from $i+1$ and $n-1$ to find if any pair sums to $target - arr[i]$. If the current sum is too small, move lo right; if too large, move hi left.

Time Complexity:

- Sorting $\rightarrow O(n \log n)$
- Outer loop $\times$ two-pointer scan $\rightarrow O(n^2)$
- **Overall $\rightarrow O(n^2)$**

The screenshot shows the GeeksforGeeks website interface for the 'Row with max 1s' problem. The left sidebar indicates the problem was solved successfully with 11/11 test cases passed, 100% accuracy, 4/4 points scored, and a time taken of 0.57. The right pane displays the Python code for the solution, which uses a greedy approach to find the row with the maximum number of 1s by moving left from the top-right corner.

```
1 class Solution:
2     def rowWithMax1s(self, arr):
3         n, m = len(arr), len(arr[0])
4         col, result = m - 1, -1
5
6         for row in range(n):
7             while col >= 0 and arr[row][col] == 1:
8                 result = row
9                 col -= 1
10
11         return result
```

Logic Used: Start from the **top-right corner** ($0, m-1$). For each row, move left as long as the current cell is 1, recording the row index. Since rows are sorted, once the column pointer moves left it never needs to go right again — each row either extends the leftmost 1 boundary or doesn't.

Time Complexity:

- Row pointer moves down n steps, column pointer moves left at most m steps total.
- **Overall** $\rightarrow O(n + m)$